

Verifier-Assisted versus LLM-Only Pre-deployment Network-Change Validation: A Preregistered Paired Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We measured whether integrating a deterministic pre-deployment verifier with a bounded automated repair budget increases the fraction of intent-driven network-change proposals that pass semantic validation compared with accepting a single LLM candidate. In a preregistered paired design (study id `cnsmp2026-llm-verifier-predeployment-001`) we executed 300 paired intents (100 per difficulty stratum: low/medium/high) using `gpt-5-mini` and a deterministic semantic validator. Each paired intent produced one shared_initial generation that was evaluated under two conditions: baseline (accept single shot) and guarded (deterministic validation and ≤ 1 automated repair). Outcomes: baseline 299/300 unflagged passes (0.9966667), guarded 300/300 (1.0); contingency $n_{11}=299, n_{10}=1, n_{01}=0, n_{00}=0$; absolute paired difference = 0.0033333; exact McNemar two-sided $p = 1.0$; 95% paired bootstrap percentile CI = [0.00,0.01] (bootstrap_seed = 1729, B = 10000). Under the supplied synthetic suite and repair budget the single-shot generator was near ceiling, limiting observable deterministic rescue. We archive preregistration, execution, and analysis artifacts and interpret results with respect to estimand conditioning, validator coverage, repair budget, and operational deployment policies.

I. INTRODUCTION

Generative large language models (LLMs) are being applied to network operations (NetOps) tasks such as intent→configuration translation, configuration synthesis, and automated repair. For pre-deployment network changes, semantic correctness properties—sequencing constraints, reachability, and policy preservation—are operationally critical and cannot be assured by superficial syntactic checks alone. A common engineering pattern is to pair generative models with deterministic validators and bounded automated repair (generate→validate→repair) to reduce operational risk. However, the measurable benefit of such guarded pipelines depends on the experimental estimand (what is held fixed), the generator single-shot quality, the validator’s expressiveness, and the permitted repair budget.

We preregistered a paired experiment (study id `cnsmp2026-llm-verifier-predeployment-001`) to quantify the deterministic rescue achievable when the same single LLM candidate is evaluated under two treatments. For each intent the pipeline produced exactly one shared_initial candidate using the declared generator (`gpt-5-mini`) and compared: (a) baseline — accept the single candidate without repair, and (b) guarded — run a deterministic validator and allow at most one automated repair which itself is re-validated. The primary research question: does adding a deterministic verifier plus at most

one automated repair increase the probability that a single LLM candidate passes semantic pre-deployment validation compared with accepting that same candidate without repair?

Contributions of this artifact-grounded study: (1) a preregistered paired evaluation isolating deterministic rescue under a shared_initial estimand that conditions on a single LLM draw per intent; (2) a complete execution and analysis bundle including validator traces and the single automated repair transcript archived with the run; and (3) an evidence-grounded interpretation of estimator conditioning, ceiling effects, and operational implications for NetOps pipelines.

II. RELATED WORK

Recent surveys and system reports document rapid progress in applying LLMs to NetOps tasks including intent→configuration translation, template synthesis, and automated testing; they also emphasize tradeoffs between model capability and operational safety. Tool-augmented architectures and planner-centric frameworks permit LLMs to call external validators, simulators, or synthesizers to decompose multi-step workflows and coordinate tool outputs. Empirical work shows that generate→validate→repair scaffolds can improve surface correctness metrics relative to unconstrained generation, but reported gains depend on validator coverage, task distribution, and whether pipelines permit resampling or verifier-guided regeneration.

Two methodological themes motivate our design. First, evaluations that mix deterministic validation with resampling or iterative regeneration conflate verifier effects with generator variance; isolating deterministic rescue requires conditioning on a single generator draw. Second, many publicly used NetOps benchmarks emphasize syntactic or template correctness rather than semantically rich invariants (reachability, sequencing, policy preservation), which limits operational generality. Research proposing semantic or verifier-centric benchmarks stresses the need for validators that can express sequencing and reachability constraints, but community adoption remains partial.

Complementary studies examine hybrid repair and verification, combining symbolic fault localization or formal checks with generative repair operators; success varies substantially with fault class and verifier expressiveness. Work on hallucination detection and deterministic safety nets (entropy or semantic-consistency detectors, retrieval augmen-

tation, self-reflection) demonstrates promising detection approaches but effectiveness depends on the threat model and the validator’s semantic reach. These bodies of work underline that validator design (which properties are checkable) and the allowed repair operator set primarily determine a guarded pipeline’s practical value. Our study explicitly conditions on a single `shared_initial` LLM draw and allows at most one constrained repair to quantify the bounded deterministic rescue attributable to post-generation checking alone.

III. METHODOLOGY

Preregistration and estimand. The confirmatory design (study id `cnsmp2026-llm-verifier-predeployment-001`) and the primary estimand were preregistered before the main run; the primary estimand is the absolute paired difference in the unflagged verifier-pass proportion (guarded - baseline) on $N = 300$ paired intents after applying prespecified inclusion/exclusion rules. Analysis executors and multiplicity handling for exploratory strata tests were fixed prior to observing main-run outcomes. An optional calibration pilot described in the preregistration was not executed.

Task generation and stratification. Tasks were produced deterministically into a synthetic topology suite stratified into three difficulty strata (low/medium/high). Each task manifest encodes: (i) the natural-language intent, (ii) initial and expected final device states, (iii) required_sequence constraints (ordered field changes such as `must_be_down_for_fields`), (iv) `preserve_unspecified_state` invariants, and (v) `protected_interfaces` that must not be modified. Difficulty metadata (`changed_interface_count`, `dependency_rule_count`, `required_command_count`) determined stratum membership and ensured equal per-stratum sampling (100 pairs per stratum). Representative difficulty patterns include `'shutdown_configure_restore'` (medium) and `'paired_link_atomic_mtu_change'` (hard).

`Shared_initial` generation protocol and model configuration. To isolate deterministic validator/repair rescue from generator resampling effects we implemented the `shared_initial` estimand: for each pair the pipeline produced exactly one `shared_initial` candidate which was reused verbatim for both baseline and guarded conditions. The declared generator for `shared_initial` production was `gpt-5-mini` with `max_output_tokens = 1500` and `reasoning_effort = "minimal"`. Reusing the same candidate across conditions prevents differences due to independent resampling and measures only deterministic validation and bounded repair applied to the identical output.

Deterministic validator, scoring rule, and repair operator. Semantic evaluation used a deterministic validator implementing the preregistered `full_intent_constraint_validation` rule. The validator deterministically checks sequencing invariants (e.g., `must_be_down_for_fields`), `required_command_count`, preservation of unspecified state, encoded topology reachability invariants, and forbidden-template protections for `protected_interfaces`. Scoring is binary: an unflagged candidate is considered a pass for the estimand. The guarded pipeline

permits at most one automated repair attempt (repair budget ≤ 1). The repair operator is a constrained edit routine limited to localized sequencing and insertion edits (for example, inserting an explicit `'admin down'` before a VLAN/MTU change) and is applied only if the validator flags the `shared_initial` candidate; repair attempts are logged and re-validated deterministically. Repair candidates that remain flagged after the permitted edit count are counted as failures for the guarded condition in the primary analysis.

Contamination controls and inclusion/exclusion rules. The preregistration included deterministic contamination detection, negative controls, and inclusion/exclusion rules specifying that a pair would be excluded only if both pipeline calls for that pair failed due to provider/infrastructure outage; pairs with a single call failure would be retained under the prespecified `complete_pair_primary` policy. In the confirmatory run no dual-call outages occurred and no pairs were excluded.

Execution instrumentation and determinism checks. The confirmatory run enforced the `shared_initial` protocol and recorded execution accounting: `planned_episode_count = 600`, `completed_episode_count = 600`, `complete_pair_count = 300`, and `model_calls_used = 301` (300 `shared_initial` generations plus one repair invocation). Provider auditing recorded `cache_miss_count = 301` and stage_counts indicating `shared_initial_generation = 300` and `repair = 1`. Deterministic reconciliation verified episode accounting and pair consistency and reported all deterministic consistency checks passed. These instrumentation values enable independent verification that the `shared_initial` protocol and the repair budget were enforced.

Analysis executor and inferential procedures. The preregistered confirmatory analysis computed contingency counts ($n_{11}, n_{10}, n_{01}, n_{00}$), the absolute paired difference (guarded - baseline), an exact two-sided McNemar test, and a paired bootstrap percentile 95% confidence interval. The bootstrap parameters used were `seed = 1729` and `resamples B = 10000`. Preplanned sensitivity analyses included treating single-call failures as failures, stratum-wise paired tests with multiplicity control, and contamination checks; these were executed as exploratory artifacts and archived accordingly.

Operational constraints. The run respected the prespecified `maximum_model_calls` and the repair budget (≤ 1). The preregistered stopping rule (halt if $>20\%$ of attempted pairs were excluded due to dual-call outages) did not trigger. All operational constraints were enforced by the execution adapter and logged in the run manifest.

Reproducibility and limits of conditioning. Design artifacts (preregistration, task manifests, model configuration, validator and repair traces, execution logs, and analysis inputs) were archived to permit deterministic replay of the confirmatory executor. Methodologically, the `shared_initial` estimand intentionally conditions on a single LLM draw per pair and therefore excludes any verifier benefit that would arise from permitting independent alternative LLM candidates per condition (resampling or iterative generation). This distinction is central to interpreting the confirmatory outcomes and is encoded in

the preregistered estimand and executor configuration.

IV. RESULTS

Execution integrity and accounting. The confirmatory run completed with `completed_episode_count` = 600, yielding `complete_pair_count` = 300 paired observations under the `shared_initial` protocol. Provider/audit records show `model_calls_used` = 301, `cache_miss_count` = 301, and `stage_counts` indicating `shared_initial_generation` = 300 and `repair` = 1. Deterministic reconciliation confirmed consistent episode accounting and the absence of provider/infrastructure failures or excluded pairs.

Primary confirmatory outcomes. Binary scoring produced `baseline_success_count` = 299 and `guarded_success_count` = 300, i.e., baseline = 299/300 (0.9966666667) and guarded = 300/300 (1.0). The paired contingency table is $n_{11} = 299$, $n_{10} = 1$, $n_{01} = 0$, $n_{00} = 0$. The preregistered exact two-sided McNemar test returned $p = 1.0$. The absolute paired improvement (guarded - baseline) equals 0.0033333333. The paired bootstrap percentile 95% confidence interval (seed = 1729, $B = 10000$) is [0.00, 0.01]. Because discordance is minimal ($m = n_{10} + n_{01} = 1$), the exact test has limited sensitivity and the bootstrap CI bounds the plausible absolute improvement to at most one percentage point in this experimental context.

Per-stratum diagnostics. Per-stratum tallies were: low difficulty baseline = 100/100 and guarded = 100/100; medium baseline = 100/100 and guarded = 100/100; high baseline = 99/100 and guarded = 100/100. All discordant evidence is concentrated in the high stratum (one rescue), while low and medium strata were at ceiling under both conditions.

Repair diagnostic and episode-level trace. The single discordant pair invoked the permitted automated repair. Validator traces attribute the initial failure to a sequencing invariant violation (the `must_be_down_for_fields` rule): the `shared_initial` candidate omitted an explicit 'admin down' step prior to a VLAN/MTU change required by the intent. The constrained repair operator performed a single localized insertion of the missing sequencing command, after which the deterministic validator reported the corrected candidate as valid. The repair therefore produced a deterministic rescue with one additional repair stage call. The run archive contains the low-level repair provider trace and the validator's before/after traces for that episode; the semantic summary above reports the substantive diagnostic.

Contamination, missingness, and negative controls. Contamination checks produced no contamination flags. The missingness summary reports zero failed or unscorable episodes and zero excluded pairs under the preregistered rules. Negative controls included by the preregistration did not cause unexpected pass behavior in the main run.

Cost and operational footprint. The guarded pipeline's deterministic rescue required a single additional repair stage call across 300 pairs (`model_calls_used` = 301), indicating that bounded automated repair under the chosen edit operator imposed minimal extra model-call cost in this run. The empirical `repair_stage_count` = 1 quantifies the low marginal

runtime/model cost for deterministic rescue under the chosen repair budget and edit operator design.

Statistical interpretation and limitations. The confirmatory inferential outcome ($p = 1.0$) primarily reflects the paucity of discordant pairs under the `shared_initial` estimand and chosen synthetic suite. Exact McNemar inference depends only on n_{10} and n_{01} ; with $n_{10} = 1$ and $n_{01} = 0$ the two-sided exact p equals 1.0, indicating no statistically detectable difference under conventional two-sided testing in this sample. The paired bootstrap CI provides a complementary bound for effect size: the absolute paired improvement is plausibly between 0 and 0.01 given observed data and the preregistered estimand. These inferences apply strictly to the `shared_initial` conditioning (one LLM draw reused across conditions), the limited repair budget (≤ 1 constrained edit), the synthetic task distribution, and the declared generator and validator.

Practical diagnostic takeaway. Under operational policies that restrict model calls per change request, the guarded generate→validate→repair pattern produced a deterministic rescue in one hard example with negligible additional model-call expenditure in this synthetic suite. However, because baseline single-shot generation was near ceiling (299/300), the marginal observable benefit of bounded deterministic repair was correspondingly small in absolute terms.

V. DISCUSSION

Estimand conditioning and operational alignment. Conditioning on a single LLM draw per intent corresponds to operational policies that constrain model calls for cost, latency, or governance reasons. Under such policies the guarded pipeline measures only deterministic validator and bounded repair rescue applied to the same candidate. Workflows that permit resampling, verifier-guided generation, or interactive steering change the estimand and typically increase achievable success rates by exploiting generator variance; those workflows must be evaluated under a different preregistered estimand.

Ceiling effects and calibration. The preregistration targeted detection of a 10 percentage-point absolute improvement with planned power ≈ 0.8 based on a pilot to estimate discordance; the optional pilot was not executed. The main run confronted a near-ceiling baseline (299/300), producing minimal discordance ($m = 1$) and reduced power to detect small absolute differences. This practical outcome highlights the utility of small calibration pilots when generator performance may approach ceiling on chosen task suites.

Validator coverage and failure-mode dependence. The observed single failure mode was a sequencing omission; other classes (reachability violations, forbidden ACL removal) were not observed at scale on this synthetic suite. The marginal utility of a guarded pipeline therefore depends on (a) the deployment distribution of failure modes and (b) validator expressiveness: richer validators that detect reachability or ACL semantics may increase observed rescue rates. Conversely, if operator risk policies disallow automated edits, the guarded pipeline's deterministic rescue becomes operationally limited to detection and human escalation.

Repair budget and bounded edits. We intentionally limited automated repair to a single constrained edit to measure bounded deterministic rescue. That design measured only localized sequencing corrections; iterative repair, larger edit operators, or interactive verifier-guided generation would increase rescue opportunities but address a different estimand. The single repair trace archived with the run enables forensic analysis of edit efficacy and could inform permitted edit-operator design in production pipelines.

Operational implications for NetOps CI/CD. Organizations must align evaluation estimands with operational choices: if pipelines permit multiple model calls or verifier-guided regeneration, evaluations should allow resampling and interactive steering to measure achievable success rates under those policies. If model calls are tightly budgeted, the `shared_initial` estimand provides an operationally meaningful upper bound for deterministic verifier/repair rescue applied to a single candidate. In this run the guarded pattern produced measurable rescue at very low additional model cost for one rescued episode out of 300.

VI. LIMITATIONS

- Synthetic benchmark: tasks and topologies were generated by the preregistered deterministic task generator; real-world heterogeneity may expose different failure modes and increase verifier marginal utility.
- Single model and validator: only gpt-5-mini and the evaluated deterministic validator were used; results may not generalize to other LLMs or richer/diverse validators.
- Ceiling effect: baseline single-shot performance was near 100% on this suite (299/300), producing limited discordance and reduced power to detect small absolute improvements relative to larger pre-specified targets.
- Pilot not executed: the preregistered pilot intended for discordance estimation and power calibration was not run; no post-preregistration power recalculation was performed.
- Estimand conditioning: the `shared_initial` protocol conditions the estimand on a single LLM candidate and therefore does not measure verifier benefit that would arise from allowing independent alternative LLM candidates per condition (resampling or iterative generation).
- Repair budget: the guarded condition permitted at most one automated repair per pair; larger or iterative repair strategies may yield different outcomes.

VII. CONCLUSION

We executed a preregistered, artifact-archived paired comparison between a verifier-assisted pipeline (deterministic validator + ≤ 1 automated repair) and accepting a single-shot LLM candidate under the `shared_initial` estimand on 300 synthetic NetOps intents using gpt-5-mini and a deterministic validator. Baseline single-shot generation produced 299/300 unflagged verifier passes and the guarded pipeline 300/300, yielding an absolute paired improvement of 0.00333 (exact McNemar two-sided $p = 1.0$; 95% bootstrap CI [0.00,0.01]).

Deterministic validator+repair rescued a single sequencing omission under the bounded repair budget. The near-ceiling baseline limits inferential sensitivity to small absolute improvements under the `shared_initial` estimand. We archive full preregistration, execution, and analysis artifacts to support reproducibility and targeted follow-up studies that vary estimand conditioning, model strength, task distribution, or repair budget.

DISCLOSURE STATEMENT

This experiment and manuscript were produced by an autonomous CNSM Agentic-AI research run executed under an archived initial master prompt (immutable SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acd1582bb39b15ba). Human role: a human Research Director selected the topic family and approved pre-lock infrastructure; all literature discovery after the autonomy lock, autonomous study selection, preregistration, execution, analysis, manuscript drafting, AI peer review, and final compilation were performed autonomously under the locked master prompt. Models and orchestration: the declared generation model used was gpt-5-mini; the execution adapter and orchestration framework recorded the guarded condition and repair budget. Validator and tooling: a deterministic semantic validator performed preregistered `full_intent_constraint_validation` checks; the guarded condition permitted at most one automated repair per pair implemented as a constrained edit operator. Preregistration: the confirmatory design, estimand, analysis executor, exclusion/missingness rules, and multiplicity plan were frozen prior to the main run and archived under the study id. Execution summary: `planned_episode_count = 600`, `completed_episode_count = 600`, `complete_pair_count = 300`, `model_calls_used = 301`; provider audit recorded one repair invocation. Pilot: the optional calibration pilot described in the preregistration was not executed. Deviations: no post-preregistration changes to the scientific design were made; no pairs were excluded. Human editing: no human manually edited or rewrote technical manuscript content after the autonomy lock. Artifact availability: all raw outputs, logs, task manifests, validator traces, repair provider traces, and analysis artifacts referenced in this manuscript are archived with the study bundle associated with the preregistered study id; the manuscript reports concise semantic summaries of archived traces rather than reproducing verbatim provider transcripts in the body.

REFERENCES

- [1] Yajie Zhou, Kevin Hsieh, Sathya Kumaran Mani, Srikanth Kandula, Zaoxing Liu, "MeshAgent: Enabling Reliable Network Management with Large Language Models", 2025, 10.1145/3771567
- [2] Xiaolong Wei, Yuehu Dong, Xingliang Wang, Xingyu Zhang, Zhejun Zhao, Dongdong Shen, Long Xia, Dawei Yin, "Beyond ReAct: A Planner-Centric Framework for Complex Tool-Augmented LLM Reasoning", 2026, 10.1609/aaai.v40i40.40676
- [3] Xu Liu, Peng Zhang, Anubhavnidhi Abhashkumar, Jiawei Chen, Weirong Jiang, "Automatic Configuration Repair", 2024, 10.1145/3696348.3696895

- [4] Hao Zhou, Chengming Hu, Ye Yuan, Yufei Cui, Yili Jin, Can Chen, Haolun Wu, Dun Yuan, Li Jiang, Di Wu, Xue Liu, Jianzhong Zhang, Xianbin Wang, Jiangchuan Liu, "Large Language Model (LLM) for Telecommunications: A Comprehensive Survey on Principles, Key Techniques, and Opportunities", 2024, 10.1109/comst.2024.3465447
- [5] X X Zhang, Xianming Gao, Peilin Tao, Tao Feng, "GraphSynth: Synthesis of Network Configuration Templates Using Large Language Models", 2025, 10.1145/3728725.3728742
- [6] Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, Yarin Gal, "Detecting hallucinations in large language models using semantic entropy", 2024, 10.1038/s41586-024-07421-0
- [7] Yunze Wei, Wei, Kaiwen, Shaohui Du, Wang, Jianyu, Liu, Zhangzhong, Yawen Wang, Zhenxin Li, Congcong Miao, Xiaohui Xie, Yong Cui, "Automated Network Protocol Testing with LLM Agents", 2025, 10.48550/arxiv.2510.13248
- [8] Fuliang Li, Haozhi Lang, Jiajie Zhang, Jiaxing Shen, Xingwei Wang, "PreConfig: A Pretrained Model for Automating Network Configuration", 2024, 10.48550/arxiv.2403.09369